МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 5

Тема: **Структурные паттерны: Фасад, Приспособленец**

Дисциплина: **«Паттерны проектирования»**

Выполнила:

Студентка группы ПИЖ-б-о-23-2
направление подготовки 09.03.04
«Программная инженерия»

Силина Оксана Романовна

Проверил:

Ассистент ДЦРСИЭ

Белик А. И.

Ставрополь 2026

Цель работы: изучить принцип работы структурных паттернов проектирования «Фасад» (Facade) и «Приспособленец» (Flyweight) и реализовать прикладную задачу расчёта траектории движения тела с использованием данных паттернов.

Задача: разработать приложение для расчёта параметров движения тела, брошенного под углом к горизонту, с использованием двух структурных паттернов проектирования.

Выбранные паттерны:

- Фасад — для объединения нескольких подсистем (валидация, расчёт модели, построение графика) в единый простой интерфейс;
- Приспособленец — для разделения внутреннего (общего) и внешнего (уникального) состояния точек траектории.

Индивидуальный вариант: 3. Разработка приложения «Траектория движения»

Репозиторий: https://github.com/tsukuri1/patterns

1. Описание паттернов

1.1. Фасад (Facade)

Фасад — это структурный паттерн проектирования, который предоставляет простой интерфейс к сложной системе классов, библиотеке или фреймворку. Фасад не запрещает обращаться к подсистеме напрямую, но предлагает удобную точку входа для типичных сценариев.

Применение в задаче:

Подсистема расчёта состоит из нескольких классов: Validator (проверка входных данных), ModelCalculator (расчёт модели), PlotBuilder (построение графика). Класс TrajectoryFacade объединяет их в один метод calculate_and_show(v0, angle), который выполняет всю цепочку действий. Клиентскому коду (интерфейсу) достаточно вызвать один метод фасада — он не знает, сколько классов задействовано внутри.

1.2. Приспособленец (Flyweight)

Приспособленец — это структурный паттерн проектирования, который позволяет экономить память, разделяя общее состояние между множеством объектов. Внутреннее состояние (общее для всех) хранится в одном экземпляре, а внешнее (уникальное) передаётся извне.

Применение в задаче:

Точки траектории (TrajectoryPoint) содержат координаты (x, y) — это внешнее состояние, уникальное для каждой точки. Общие параметры расчёта (ускорение свободного падения g, коэффициент сопротивления k, шаг интегрирования) не зависят от конкретной точки — это внутреннее состояние. Оно вынесено в класс PointFactory, который создаёт и кэширует общие объекты PointStyle. При построении траектории из сотен точек общее состояние создаётся один раз, а не дублируется в каждой точке.

2. Структура приложения

Приложение состоит из следующих компонентов:

Компонент	Назначение
Validator	Проверка входных данных (подсистема Фасада)
ModelCalculator	Расчёт параметров движения (подсистема Фасада)
PlotBuilder	Построение графика (подсистема Фасада)
TrajectoryFacade	Единый интерфейс к подсистеме (Фасад)
PointStyle	Общее (внутреннее) состояние точек (Приспособленец)
TrajectoryPoint	Точка траектории с внешним состоянием
PointFactory	Фабрика и кэш общих стилей (Приспособленец)
TrajectoryApp	Графический интерфейс приложения

3. Диаграмма классов

Для отображения структуры разработанного приложения была создана UML-диаграмма классов.

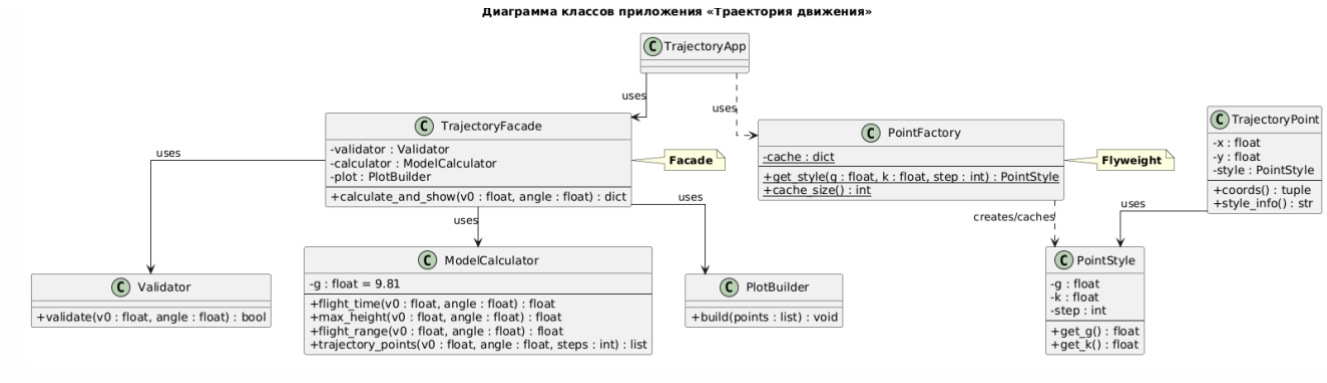


Рисунок 1 — Диаграмма классов приложения «Траектория движения»

Диаграмма содержит:

- Класс TrajectoryFacade и подсистему Validator, ModelCalculator, PlotBuilder (паттерн «Фасад»);
- Классы PointStyle, TrajectoryPoint, PointFactory (паттерн «Приспособленец»);
- Класс TrajectoryApp — интерфейс приложения.

4. Реализация паттернов

4.1. Паттерн «Фасад»

TrajectoryFacade объединяет три подсистемы: Validator проверяет входные данные, ModelCalculator выполняет расчёт, PlotBuilder строит график. Клиентский код вызывает только calculate_and_show() и не знает о внутреннем устройстве подсистемы. Это позволяет менять реализацию отдельных модулей, не затрагивая интерфейс приложения.

```
import math


class Validator:
    def validate(self, v0, angle):
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
```

```
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        return True


class ModelCalculator:
    def __init__(self, g=9.81):
        self.g = g

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / self.g

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / self.g

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class PlotBuilder:
    def build(self, points):
        import matplotlib.pyplot as plt
        xs = [p[0] for p in points]
        ys = [p[1] for p in points]
        plt.plot(xs, ys, marker='o', markersize=3)
        plt.title("Траектория движения")
        plt.xlabel("X, м")
        plt.ylabel("Y, м")
        plt.grid(True)
        plt.axis('equal')
        plt.show()
```

```
class TrajectoryFacade:
    """Фасад: простой интерфейс к подсистеме расчёта."""
    def __init__(self):
        self.validator = Validator()
        self.calculator = ModelCalculator()
        self.plot = PlotBuilder()

    def calculate_and_show(self, v0, angle):
        self.validator.validate(v0, angle)
        T = self.calculator.flight_time(v0, angle)
        H = self.calculator.max_height(v0, angle)
        L = self.calculator.flight_range(v0, angle)
        pts = self.calculator.trajectory_points(v0, angle, 50)
        self.plot.build(pts)
        return {"time": T, "height": H, "range": L, "points": pts}
```

4.2 Паттерн «Приспособленец»

PointStyle хранит общее (внутреннее) состояние: параметры g, k, шаг. TrajectoryPoint хранит уникальное (внешнее) состояние — координаты (x, y) — и ссылку на общий PointStyle. PointFactory кэширует стили по ключу, чтобы при построении сотен точек один и тот же стиль не создавался заново.

```
class PointStyle:
    """Внутреннее состояние: общее для всех точек."""
    def __init__(self, g, k, step):
        self.g = g
        self.k = k
        self.step = step

    def get_g(self):
        return self.g

    def get_k(self):
        return self.k


class TrajectoryPoint:
    """Точка траектории: внешнее состояние + ссылка на общий стиль."""
```

```
    def __init__(self, x, y, style: PointStyle):
        self.x = x
        self.y = y
        self.style = style

    def coords(self):
        return (self.x, self.y)

    def style_info(self):
        return f"g={self.style.get_g()}, k={self.style.get_k()}"


class PointFactory:
    """Фабрика и кэш общих стилей (Flyweight)."""
    _cache = {}

    @staticmethod
    def get_style(g, k, step):
        key = (g, k, step)
        if key not in PointFactory._cache:
            PointFactory._cache[key] = PointStyle(g, k, step)
        return PointFactory._cache[key]

    @staticmethod
    def cache_size():
        return len(PointFactory._cache)
```

4.3. Графический интерфейс

Интерфейс построен так, чтобы обе возможности паттернов были видны пользователю. Кнопка «Рассчитать» вызывает один метод фасада calculate_and_show() — клиентский код не знает, что внутри работают три подсистемы. Кнопка «Показать кэш приспособленцев» демонстрирует, что общие стили точек создаются один раз и переиспользуются: при повторных расчётах с теми же параметрами размер кэша не растёт.

```
import tkinter as tk
from tkinter import messagebox


class TrajectoryApp:
    def __init__(self, root):
```

```
        self.root = root
        root.title("Траектория движения")

        self.facade = TrajectoryFacade()

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)

        tk.Button(root, text="Рассчитать",
                  command=self.calculate).grid(row=2,          column=0,
columnspan=2, pady=5)
        tk.Button(root, text="Показать кэш приспособленцев",
                  command=self.show_cache).grid(row=3,         column=0,
columnspan=2)

        self.result = tk.Text(root, width=50, height=12)
        self.result.grid(row=4, column=0, columnspan=2)

    def _read_inputs(self):
        v0 = float(self.v0_entry.get())
        angle = float(self.angle_entry.get())
        return v0, angle

    def calculate(self):
        try:
            v0, angle = self._read_inputs()
            res = self.facade.calculate_and_show(v0, angle)

            # Демонстрация приспособленца: точки используют общий
стиль
            style = PointFactory.get_style(9.81, 0.0, 50)
            points = [TrajectoryPoint(x, y, style) for (x, y) in res["points"]]

            self.result.delete("1.0", tk.END)
            self.result.insert(tk.END,
                f"Время полёта: {res['time']:.3f} c\n"
```

```
                f"Макс. высота: {res['height']:.3f} м\n"
                f"Дальность: {res['range']:.3f} м\n"
                f"Точек траектории: {len(points)}\n"
                f"Стиль точки (пример): {points[0].style_info()}\n"
                f"Размер кэша стилей: {PointFactory.cache_size()}\n")
        except ValueError as e:
            messagebox.showerror("Ошибка ввода", str(e))
        except Exception as e:
            messagebox.showerror("Ошибка", str(e))

    def show_cache(self):
        messagebox.showinfo("Приспособленец",
                            f"В кэше {PointFactory.cache_size()} общих стилей.\n"
                            f"Все точки траектории используют один и тот же объект PointStyle.")
```

5. Результаты работы программы

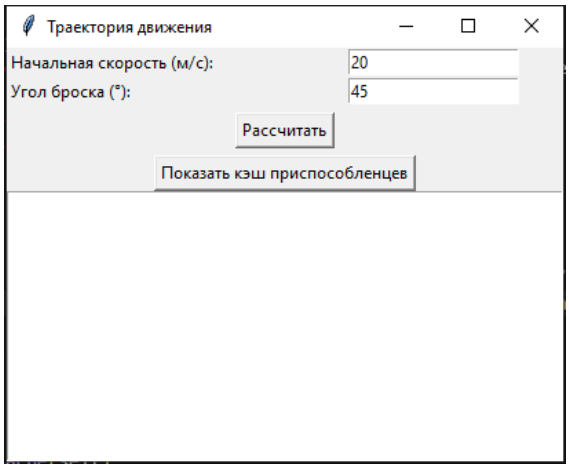


Рисунок 2 — Главное окно приложения

Пользователь вводит начальную скорость и угол броска, затем нажимает «Рассчитать». Один вызов фасада запускает всю цепочку: валидацию, расчёт, построение графика.

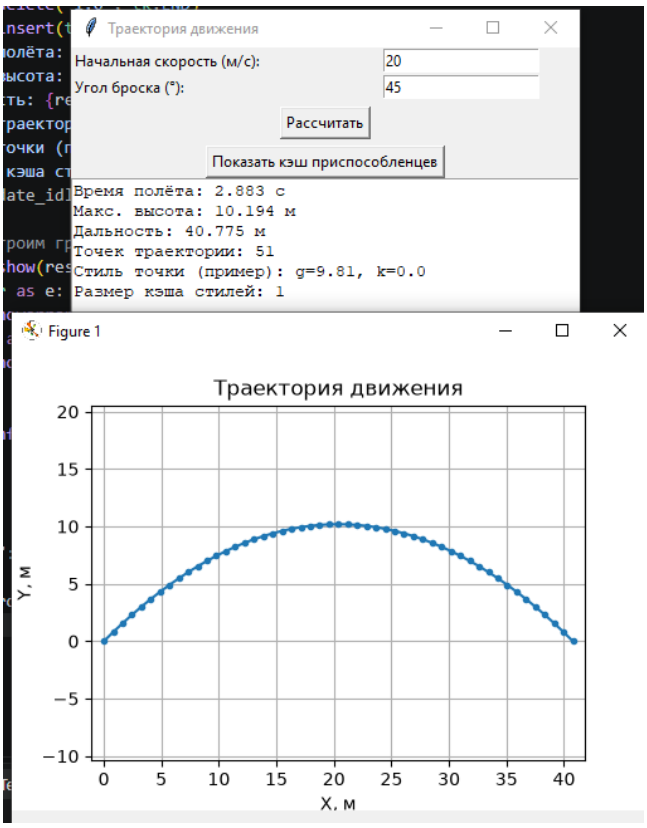


Рисунок 3 — Расчёт траектории через фасад

При v0 = 20 м/с, угол = 45°:

- Время полёта: 2.883 с
- Макс. высота: 10.194 м
- Дальность: 40.774 м
- Точек траектории: 51
- Размер кэша стилей: 1

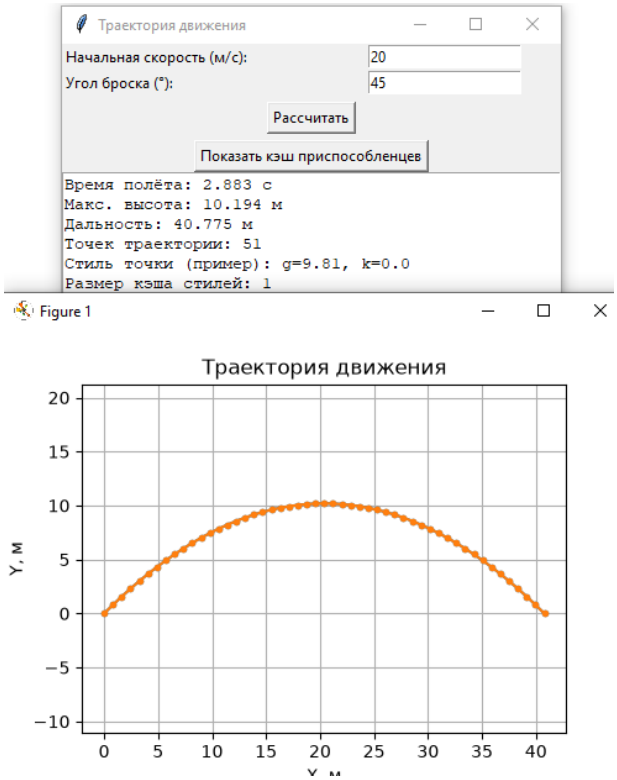


Рисунок 4 — Повторный расчёт с теми же параметрами

При повторном нажатии «Рассчитать» размер кэша остаётся равным 1: общий стиль точек не создаётся заново, а переиспользуется. Это и демонстрирует экономию памяти приспособленцем.

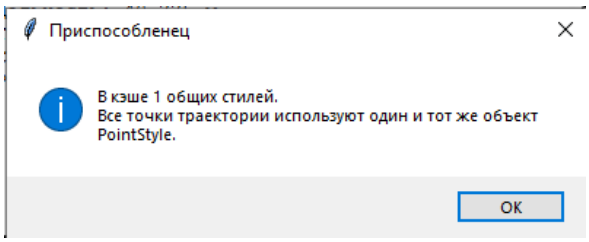


Рисунок 5 — Окно «Показать кэш приспособленцев»

Появляется сообщение: «В кэше 1 общих стилей. Все точки траектории используют один и тот же объект PointStyle».

6. Исходный код

```
import math
```

```
import tkinter as tk
from tkinter import messagebox


class Validator:
    def validate(self, v0, angle):
        if v0 <= 0:
            raise ValueError("Скорость должна быть > 0")
        if not (0 < angle < 90):
            raise ValueError("Угол должен быть в диапазоне (0; 90)")
        return True


class ModelCalculator:
    def __init__(self, g=9.81):
        self.g = g

    def flight_time(self, v0, angle):
        a = math.radians(angle)
        return 2 * v0 * math.sin(a) / self.g

    def max_height(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * (math.sin(a) ** 2) / (2 * self.g)

    def flight_range(self, v0, angle):
        a = math.radians(angle)
        return (v0 ** 2) * math.sin(2 * a) / self.g

    def trajectory_points(self, v0, angle, steps):
        T = self.flight_time(v0, angle)
        a = math.radians(angle)
        pts = []
        for i in range(steps + 1):
            t = T * i / steps
            x = v0 * math.cos(a) * t
            y = v0 * math.sin(a) * t - 0.5 * self.g * t ** 2
            pts.append((round(x, 3), round(y, 3)))
        return pts


class PlotBuilder:
    def build(self, points):
        import matplotlib.pyplot as plt
        xs = [p[0] for p in points]
```

```
        ys = [p[1] for p in points]
        plt.plot(xs, ys, marker='o', markersize=3)
        plt.title("Траектория движения")
        plt.xlabel("X, м")
        plt.ylabel("Y, м")
        plt.grid(True)
        plt.axis('equal')
        plt.show()


class TrajectoryFacade:
    """Фасад: простой интерфейс к подсистеме расчёта."""
    def __init__(self):
        self.validator = Validator()
        self.calculator = ModelCalculator()
        self.plot = PlotBuilder()

    def calculate_and_show(self, v0, angle):
        self.validator.validate(v0, angle)
        T = self.calculator.flight_time(v0, angle)
        H = self.calculator.max_height(v0, angle)
        L = self.calculator.flight_range(v0, angle)
        pts = self.calculator.trajectory_points(v0, angle, 50)
        self.plot.build(pts)
        return {"time": T, "height": H, "range": L, "points": pts}


class PointStyle:
    """Внутреннее состояние: общее для всех точек."""
    def __init__(self, g, k, step):
        self.g = g
        self.k = k
        self.step = step

    def get_g(self):
        return self.g

    def get_k(self):
        return self.k


class TrajectoryPoint:
    """Точка траектории: внешнее состояние + ссылка на общий стиль."""
    def __init__(self, x, y, style: PointStyle):
```

```
        self.x = x
        self.y = y
        self.style = style

    def coords(self):
        return (self.x, self.y)

    def style_info(self):
        return f"g={self.style.get_g()}, k={self.style.get_k()}"


class PointFactory:
    """Фабрика и кэш общих стилей (Flyweight)."""
    _cache = {}

    @staticmethod
    def get_style(g, k, step):
        key = (g, k, step)
        if key not in PointFactory._cache:
            PointFactory._cache[key] = PointStyle(g, k, step)
        return PointFactory._cache[key]

    @staticmethod
    def cache_size():
        return len(PointFactory._cache)


class TrajectoryApp:
    def __init__(self, root):
        self.root = root
        root.title("Траектория движения")

        self.facade = TrajectoryFacade()

        tk.Label(root, text="Начальная скорость (м/с):").grid(row=0,
column=0, sticky="w")
        self.v0_entry = tk.Entry(root)
        self.v0_entry.insert(0, "20")
        self.v0_entry.grid(row=0, column=1)

        tk.Label(root, text="Угол броска (°):").grid(row=1, column=0,
sticky="w")
        self.angle_entry = tk.Entry(root)
        self.angle_entry.insert(0, "45")
        self.angle_entry.grid(row=1, column=1)
```

```
            tk.Button(root, text="Рассчитать",
                      command=self.calculate).grid(row=2, column=0,
columnspan=2, pady=5)
            tk.Button(root, text="Показать кэш приспособленцев",
                      command=self.show_cache).grid(row=3, column=0,
columnspan=2)

            self.result = tk.Text(root, width=50, height=12)
            self.result.grid(row=4, column=0, columnspan=2)

        def _read_inputs(self):
            v0 = float(self.v0_entry.get())
            angle = float(self.angle_entry.get())
            return v0, angle

        def calculate(self):
            try:
                v0, angle = self._read_inputs()
                res = self.facade.calculate_and_show(v0, angle)

                style = PointFactory.get_style(9.81, 0.0, 50)
                points = [TrajectoryPoint(x, y, style) for (x, y) in res["points"]]

                self.result.delete("1.0", tk.END)
                self.result.insert(tk.END,
                    f"Время полёта: {res['time']:.3f} с\n"
                    f"Макс. высота: {res['height']:.3f} м\n"
                    f"Дальность: {res['range']:.3f} м\n"
                    f"Точек траектории: {len(points)}\n"
                    f"Стиль точки (пример): {points[0].style_info()}\n"
                    f"Размер кэша стилей: {PointFactory.cache_size()}\n")
            except ValueError as e:
                messagebox.showerror("Ошибка ввода", str(e))
            except Exception as e:
                messagebox.showerror("Ошибка", str(e))

        def show_cache(self):
            messagebox.showinfo("Приспособленец",
                                f"В кэше {PointFactory.cache_size()} общих
стилей.\n"
                                f"Все точки траектории используют один и тот
же объект PointStyle.")
```

```
    if __name__ == "__main__":
        root = tk.Tk()
        app = TrajectoryApp(root)
root.mainloop()
```

Вывод:

В ходе выполнения лабораторной работы было разработано приложение «Траектория движения», реализующее расчёт параметров движения тела, брошенного под углом к горизонту.

В приложении применены два структурных паттерна проектирования:

- Фасад — для объединения трёх подсистем (Validator, ModelCalculator, PlotBuilder) в единый интерфейс TrajectoryFacade. Клиентский код вызывает один метод calculate_and_show() и не зависит от внутреннего устройства подсистемы.
- Приспособленец — для разделения общего и уникального состояния точек траектории. Общие параметры (g, k, шаг) хранятся в одном экземпляре PointStyle, который кэшируется фабрикой PointFactory; координаты (x, y) остаются уникальными для каждой точки. Это позволяет экономить память при построении траектории из большого числа точек.

Таким образом, была изучена практическая реализация двух структурных паттернов и получено работоспособное приложение с графическим интерфейсом для расчёта и визуализации траектории движения тела.